

Practicum in Operating Systems

Course orientation

Ondřej (Ondra) Krajiček

me@ondrejkrasicek.com · linkedin.com/in/OndrejKrajsicek

Built 2026-09-15 16:13 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

What we'll cover

This course grades you on what you can see a system do. It does not grade you on what you can recite about it.

1	Who and why	The gap a practicum fills beside your theory courses
2	The topics	Eight topic areas, and what each one lets you do
3	Assessment	Two components, three assignments, real dates, who grades them
4	Reading	Four people who write well about this material
5	Your machine	What to install before the first lab

The payoff is part 3. Passing an assignment removes its subject from your exam.

PART I

Why this course exists

The gap between what you can define and what you have seen.

Who runs this course

Ondřej Krajíček	krajicek@mail.muni.cz — lecturer
------------------------	----------------------------------

Co-lecturer	named in IS MUNI, with contact details, once the course opens
--------------------	---

Assisting with the course: **three teaching assistants**, whom you will meet in the labs. IS MUNI carries the roster.

Material comes **from IS MUNI**. Work goes **to FI MUNI GitLab**, at `gitlab.fi.muni.cz`. Two systems, two directions.

Write to either of us. You do not need a reason, and you do not need to wait for a seminar.

You can define a system call. You have not seen one.

What you have

You can write Python. You can write C. You can define a process, a system call and virtual memory.

What you have not

You have never attached a tracer to a running program and read what it actually did.

This is not a gap in you. It is a gap in the curriculum, and it dates from a licence change in 1979.
[S5]

[S5] Lessons Learned from 30 Years of MINIX — Tanenbaum, CACM (2016). <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>

Operating systems went back to theory in 1979

In 1977 John Lions wrote a line-by-line commentary on the UNIX version 6 kernel. Hundreds of universities taught from it. [S5]

The 1979 AT&T release changed the licence and forbade teaching the source. Courses **went back to theory-only mode** or moved to toy simulators. [S5]

The teaching stayed theoretical [S5]. A disabled DTrace probe costs nothing, which is "what made DTrace safe to ship enabled on production systems" [V23]. You will attach one to a running program.

[S5] Lessons Learned from 30 Years of MINIX — Tanenbaum, CACM (2016). <https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf> ·

[V23] Observing a Running System

Looking and reciting are different skills

Reciting

You answer a question somebody already wrote down. A monitoring dashboard does this: "it answers questions you already knew to ask" [V2]

Looking

You ask a question nobody wrote down in advance, and the running system answers it. The field calls this observability: "the ability to investigate a system by asking any question about its behavior without needing to predict those questions in advance" [V1]

This course grades the second one. You can already recite more than you can predict.

Thirty seconds, silently, to yourself

In your head, say the steps between a Python `f.write()` returning and the bytes being on the disk.

Rozenblit and Keil measured what happens next. People rate their own understanding **lower after they try to explain** the thing than before. [S1]

If your explanation had a gap, that gap is the course. Nobody is graded on this, and nobody has to say it aloud.

[S1] The misunderstood limits of folk science: an illusion of explanatory depth — Rozenblit & Keil, Cognitive Science 26 (2002).
cogdevlab.yale.edu

What a practicum asks of you

A lecture course asks you to reproduce an explanation. A practicum asks you to produce an observation and defend it.

Crouch and Mazur measured the difference in a physics classroom. Students who only watched a demonstration gained little understanding of the concept it showed. Students who **predicted the outcome first** understood that concept measurably better. [S2]

So I will not prove the thesis in this lecture. The next lecture opens with a three-line program whose output disappears.

[S2] Classroom demonstrations: learning tools or entertainment? — Crouch, Fagen, Callan & Mazur, Am. J. Phys. 72 (2004).
<https://works.swarthmore.edu/fac-physics/203/>

Takeaways

- 1 The course grades looking**
Not reciting. The assessment in part 3 is built around that, and so is every topic
- 2 It works downward**
From the code you write to the operating system under it. Each topic takes one thing you already use and shows the mechanism

Two claims. If you remember nothing else from this half hour, remember these.

PART II

The course

Eight topic areas, and one idea that carries all of them.

One idea carries all eight topics

Every operating-system interface keeps its promise only under conditions nobody wrote down.

The contract The written half says what the call does. The scope clause says when it does that. [V4]

The method You cannot reason out which layer lies. Record the boundary crossings. Then count them. [V5]

A leak is your program leaving the scope, not the abstraction failing [V4]. A library's documentation is a claim. The system call it issues is a fact [V21]. Trust the trace when they disagree.

[V4] [V5] Where the Abstraction Leaks · [V21] AI Agents as Debugging Operators

Topic 1— I/O: files and the network

What it covers	One file write and one network request, to the device
The general rule	Nearly every fast call returns before the work happens. "Almost every fast OS interface returns when the kernel has taken responsibility for the work, not when the work has happened." [V6]
The divergence	On Linux a socket is a file descriptor. On Windows it is not: "In Winsock applications, a socket descriptor is not a file descriptor and must be used with the Winsock functions." [V7]

The course starts here: following one write down makes a tracer necessary.

[V6] Where the Abstraction Leaks · [V7] Sockets and Network I/O

Topic 2 — The system-call boundary

What it covers	Where your program stops and the operating system begins, and how you cross that line
The divergence	Linux publishes the system call interface. Windows publishes Win32 and reserves the layer below it [V8]
The correction	"Both are stable at the layer they nominate. The disagreement is about which layer is nominated" [V8]

You will be able to name where the boundary sits on each platform, and say why they chose differently.

[V8] Windows and Linux place the stable syscall boundary at different layers — author's working notes, summarised in the handout appendix.

Topic 3 — Observing a running system

What it covers	strace on Linux, NtTrace on Windows, and how to read what they print
The honest limit	A tracer adds code, so it changes the timing it reports. "A tracer is not a window onto a running program. It is an edit to that program." [V22]
The precedent	A published graduate course "teaches kernel behaviour through DTrace and hardware performance counters" [V9] [S3]

You will be able to state a tracer's cost and its blind spot.

[S3] Teaching OS with FreeBSD through Tracing — Neville-Neil et al. (2017).

https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf · [V22] [V25] Tracing cost and blind spot

Topic 4 — Virtual memory and the page cache

What it covers	Where the bytes you wrote went, and why freed memory is still held
The mechanism	An allocation gives you address space, not memory. "A mapping can exist with no physical page behind it. The page is materialized on first touch, by a fault." [V17]
The consequence	One process yields several memory figures, each measuring a different thing. "How much memory does this process use" turns out to be a badly posed question [V17]

You will be able to read a memory figure and say which question it answers.

[V17] Virtual Memory and the Page Cache — author's working notes, summarised in the handout appendix.

Topic 5 — Protection, shared memory and IPC

What it covers	How one address space is isolated from another, and every way you deliberately cross that line
The mechanism	"Every load, store, and instruction fetch a process performs is checked before it completes." [V18]
Where it is checked	In hardware, by the memory management unit, against permission bits the kernel wrote [V18]

You will be able to say why a segmentation fault is the protection working rather than failing.

[V18] Memory Protection — author's working notes, summarised in the handout appendix.

Topic 6 — Processes, threads and scheduling

What it covers	What the operating system actually schedules, and how language runtimes redefine the thread you think you have
The finding	"A userspace - scheduled unit of concurrency produces no system call when it is created and no system call when it yields" [V19]
Why that matters	"a syscall tracer shows nothing at all where the student expects to see concurrency" [V19]

Here the absence is the finding. You will be able to read an empty trace as evidence.

[V19] Processes, Threads and Scheduling — author's working notes, summarised in the handout appendix.

Topic 7 — Post-mortem analysis, with and without an agent

What it covers	Reading what a process left behind, and driving the tools with an AI agent
Why it is separate	Every earlier topic needs a live process. Here it is dead: "A dump is all you get once it is already dead." [V20]
The agent question	An agent adds breadth over a large trace, and little on one stack frame [V26]. It reads only what the trace and the dump captured [V27]

You will be able to say which half of a session a model can take.

[V20] Crash Dump Analysis · [V26] [V27] AI Agents as Debugging Operators

Topic 8 — Leaks in the abstraction

The stance

Your program steps outside the conditions the abstraction covers. "The leak is not the abstraction failing; it is the program leaving the scope."
[V4]

What you take away

Your own list of scope clauses a program stepped outside of

The pace

No fixed week per topic. Three weeks are reviews built from your submissions

By then your list has entries like a write that returned before the data was safe.

[V4] Where the Abstraction Leaks — author's notes.

How you are assessed

The rules, the dates, and the argument behind them.

Two components: course credit and exam

course credit
(zápočet)

You earn it by submitting at least one of the three homework assignments. You do not have to pass one. The checker can fail your submission, and it still earns the course credit. You must pass the exam as well

exam (zkouška)

No AI of any kind in the room — no model you bring, no agent harness. Passing an assignment shrinks it

Both rules above are specific to PB158. I give the shrinking mechanism, and the reason for the AI rule, later in this part.

Three assignments, and the real dates

ASSIGNMENT	SET	DUE, MIDNIGHT	REVIEWED
1	Tue 22 Sep	Fri 2 Oct	Tue 6 Oct
2	Tue 6 Oct	Fri 16 Oct	Tue 20 Oct
3	Tue 20 Oct	Fri 30 Oct	Tue 3 Nov

Teams of two to three. Submitted to FI MUNI GitLab at `gitlab.fi.muni.cz`. Pass or fail, and no points.

Nothing is set today. The first assignment arrives next Tuesday, and you have ten days for it.

The homework is a loop, not a test

Resubmit as often as you like until the deadline. A fail costs time and nothing else.

What the checker returns

Defects, not remedies. It names what is wrong, never the fix

Why not remedies

A critic is normally told to "suggest concrete revisions" [V15]. We drop that half on purpose

Push your failures. A passing commit is by construction the one with nothing left that the checker can catch.

[V15] Generative Actor Critic — author's notes.

What passing an assignment buys you

-
- | | | |
|---|-----------------------------------|---|
| 1 | The exam has two parts | An irreducible core every student sits unaided, and a removable part |
| 2 | A pass removes its subject | Passing an assignment removes that subject from your exam. Fixed, not chosen by you |
| 3 | The whole team gets it | The submission is the team's, and the reduction applies to every member |
-

**A checker you may retry without limit is a target by construction, so it stops measuring you.
"When a measure becomes a target, it ceases to be a good measure" [V16]. That is why the
unaided core stays.**

[V16] Goodhart's Law — author's notes.

An AI grades you, and a human is on the loop

An AI grades both the exam and the homework. A human stays on the loop.

What that means

Nobody checks a grade before you get it. A human catches a wrong result afterwards: "the action does not wait for the human. It proceeds by default" [V10]

The asymmetry

You may not use AI in the exam, and an AI grades you. Named here rather than found later

[V10] Human on the Loop — author's notes.

How this arrangement fails

Vigilance decays	Mackworth's watchers hunted rare double jumps in a pointer stepping once a second, for two hours [V24]. Detection fell from about 85% to about 75% in 30 minutes [V11]
Review becomes ritual	Gate everything and the reviewer clicks through: "a rubber stamp with an audit trail" [V12]
A critic invents faults	Where models started at $\geq 75\%$ task accuracy, "The critic, primed to find errors, invented them." [V14]

So tell us when a result is wrong. "the human's job is to adjudicate a raised alarm" [V13]

[V11] [V12] [V13] [V24] Human on the Loop · [V14] Generative Actor Critic

What this course is betting

The bet: unlimited retries, with two counterweights — a checker that names defects, never remedies, and an unaided exam core.

Three published results argue against it.

Retries [S6]	A team pushes one-line edits to see which test flips. Limits and regression penalties cut that by up to 85%
Feedback [S18]	Rapid feedback leads some students to aim at the tests, not the program
Guidance [S22]	Under minimal guidance a learner with low prior knowledge learns less than under strong guidance

Edwards' counterweight was student-written tests [S17]. Ours are the two above.

[S6] <https://dl.acm.org/doi/10.1145/3591210> [S17] <https://web-cat.org/publications/pos27-Edwards.pdf> [S18] <https://valerio-terragni.github.io/assets/pdf/luxton-ace-2023.pdf> [S22] https://itgs.ict.usc.edu/papers/Constructivism_KirschnerEtAl_EP_06.pdf

Where to keep learning

-
- | | | |
|---|--------------------------|---|
| 1 | Julia Evans | <code>jvns.ca</code> — debugging and tracing from the outside in, including why <code>strace</code> fails in Docker [S12] |
| 2 | Raymond Chen | <i>The Old New Thing</i> — decades of why Windows is the way it is, still publishing [S14] |
| 3 | Brendan Gregg | flame graphs, eBPF, and the tracing overhead figures this course quotes [S13] |
| 4 | Oxide and Friends | Bryan Cantrill co-created DTrace, one of the two tracing traditions here [S16] |
-

Pick one. The handout carries the full annotated list with URLs.

[S12] <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/> · [S13] <https://www.brendangregg.com/> · [S14] <https://devblogs.microsoft.com/oldnewthing/> · [S16] <https://oxide.computer/podcasts/oxide-and-friends>

What must work on your Windows machine

TOOL	WHAT FOR
Visual Studio Community	cl.exe, the Windows SDK, PDB symbols. Tick Desktop development with C++
VS Code + C/C++ extension	the editor, on both platforms
Sysinternals Suite	Process Monitor and Process Explorer
NtTrace	native API tracing, the strace counterpart on Windows
WinDbg	crash dumps, for topic 7
WSL2	how you reach the Linux side from Windows
Python and uv	"An extremely fast Python package and project manager, written in Rust" [S21]

Microsoft native, by decision. No MinGW and no MSYS2, because WinDbg wants MSVC symbols.

What must work on your Linux machine

TOOL	WHERE
VS Code	<code>code.visualstudio.com</code> , same editor as the Windows side
gcc, make, libc headers	your distribution's build-essential equivalent
strace and gdb	your distribution's own packages
Python and uv	<code>docs.astral.sh/uv</code> [S21]

The course **dev container** carries all of this. A dev container is "a running container with a well-defined tool/runtime stack and its prerequisites" [S10]

Faculty machines exist for both platforms, so you are not blocked if your laptop refuses.

[S10] Development Containers — containers.dev. <https://containers.dev/> · [S21] uv — Astral. <https://docs.astral.sh/uv/>

Two things that cost an evening

- 1 Visual Studio Community is large** Microsoft's figure is 20 to 50 GB for a typical installation [S19]
- 2 cl.exe needs its own shell** "Even a simple Visual Studio installation can set twenty or more environment variables" [S20]

Use the Developer Command Prompt. An ordinary shell says "not recognised", which reads as a broken install rather than a missing environment.

[S19] Visual Studio 2026 system requirements · [S20] MSVC from the command line — Microsoft Learn. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>

Summary — what follows from this

- 1** **Nothing is graded on recall** The exam core measures what you can do unaided.
Homework is a loop
- 2** **Submit the attempt you doubt** The review lectures are built from your failures
- 3** **Check us** The grader has named failure modes [V11] [V12]. You
raise the alarm. A human adjudicates it [V13]

Now the second lecture: three lines of Python whose output disappears.

[V11] [V12] [V13] Human on the Loop

References • 1/7

- **[S1]** The misunderstood limits of folk science: an illusion of explanatory depth — Rozenblit & Keil, Cognitive Science 26 (2002).
<https://cogdevlab.yale.edu/sites/default/files/files/rozenblit%20&%20keil%20%202002.pdf>
- **[S2]** Classroom demonstrations: learning tools or entertainment? — Crouch, Fagen, Callan & Mazur, Am. J. Phys. 72 (2004). <https://works.swarthmore.edu/fac-physics/203/>
- **[S3]** Teaching Operating Systems with FreeBSD through Tracing, Analysis, and Experimentation — Neville-Neil et al., USENIX ;login: (2017).
https://www.usenix.org/system/files/login/articles/login_spring17_03_neville-neil.pdf
- **[S4]** On teachers working with large-language-model grading in classrooms — Tian et al. (2025), arXiv:2506.07955. Recorded title contradicted on re-reading, and no exact title confirmed in this build.
<https://arxiv.org/abs/2506.07955>

References · 2/7

- **[S5]** Lessons Learned from 30 Years of MINIX — Tanenbaum, CACM (2016).
<https://www.cs.vu.nl/~ast/Publications/Papers/cacm-2016.pdf>
- **[S6]** On submission limits and regression penalties — Lawrence, Foss & Urazova, ACM TOCE (2023).
<https://dl.acm.org/doi/10.1145/3591210>
- **[S7]** The Keller Plan and the Personalized System of Instruction — Eyre, Behav. Analyst Today 8(3) (2007). <https://files.eric.ed.gov/fulltext/EJ800986.pdf>
- **[S8]** Ironies of Automation — Bainbridge, Automatica 19(6) (1983).
<https://gvern.net/doc/sociology/technology/1983-bainbridge.pdf>
- **[S9]** MU Study and Examination Regulations — Masaryk University, official notice board, current edition. <https://www.muni.cz/en/about-us/official-notice-board/mu-study-and-examination-regulations>

References • 3/7

- **[S10]** Development Containers — containers.dev. <https://containers.dev/>
- **[S11]** Create a Dev Container — Visual Studio Code documentation. <https://code.visualstudio.com/docs/devcontainers/create-dev-container>
- **[S12]** Why strace doesn't work in Docker — Julia Evans, jvns.ca (2020). <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S13]** brendangregg.com — Brendan Gregg. <https://www.brendangregg.com/>
- **[S14]** The Old New Thing — Raymond Chen, Microsoft Developer Blogs. <https://devblogs.microsoft.com/oldnewthing/>
- **[S15]** LWN.net — weekly Linux kernel reporting. <https://lwn.net/>

References • 4/7

- **[S16]** Oxide and Friends — Bryan Cantrill and Adam Leventhal.
<https://oxide.computer/podcasts/oxide-and-friends>
- **[S17]** Web-CAT project paper, pos27-Edwards.pdf — Stephen H. Edwards, Virginia Tech. <https://web-cat.org/publications/pos27-Edwards.pdf>
- **[S18]** On rapid feedback and student submission behaviour — Luxton-Reilly, Tempero & Arachchilage, ACE '23. <https://valerio-terragni.github.io/assets/pdf/luxton-ace-2023.pdf>
- **[S19]** Visual Studio 2026 system requirements — Microsoft Learn. <https://learn.microsoft.com/en-us/visualstudio/releases/2026/vs-system-requirements>
- **[S20]** Using the MSVC toolset from the command line — Microsoft Learn.
<https://learn.microsoft.com/en-us/cpp/build/building-on-the-command-line>

References · 5/7

- **[S21]** uv — a Python package and project manager — Astral. <https://docs.astral.sh/uv/>
- **[S22]** Why Minimal Guidance During Instruction Does Not Work — Kirschner, Sweller & Clark, Educational Psychologist 41(2) (2006).
https://itgs.ict.usc.edu/papers/Constructivism_KirschnerEtAl_EP_06.pdf
- **[V1] [V2] [V3]** Observability vs Monitoring — author's working notes.
- **[V4] [V5] [V6]** Where the Abstraction Leaks — author's working notes.
- **[V7]** Sockets and Network I/O — author's working notes.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — author's working notes.

References · 6/7

- **[V9] [V23]** Observing a Running System — author's working notes.
- **[V10] [V11] [V12] [V13] [V24]** Human on the Loop — author's working notes.
- **[V14] [V15]** Generative Actor Critic as an Agentic Orchestration Pattern — author's working notes.
- **[V16]** Goodhart's Law — author's working notes.
- **[V17]** Virtual Memory and the Page Cache — author's working notes.
- **[V18]** Memory Protection — author's working notes.
- **[V19]** Processes, Threads and Scheduling — author's working notes.

References • 7/7

- **[V20]** Crash Dump Analysis on Windows and Linux — author's working notes.
- **[V21] [V26] [V27]** AI Agents as Debugging Operators — author's working notes.
- **[V22] [V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — author's working notes.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com